Polimorfism. Double Dispatch. Clase Interne

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

OOP, 2018

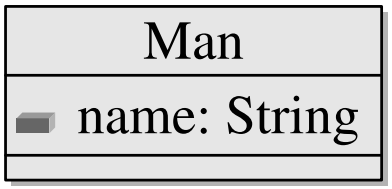




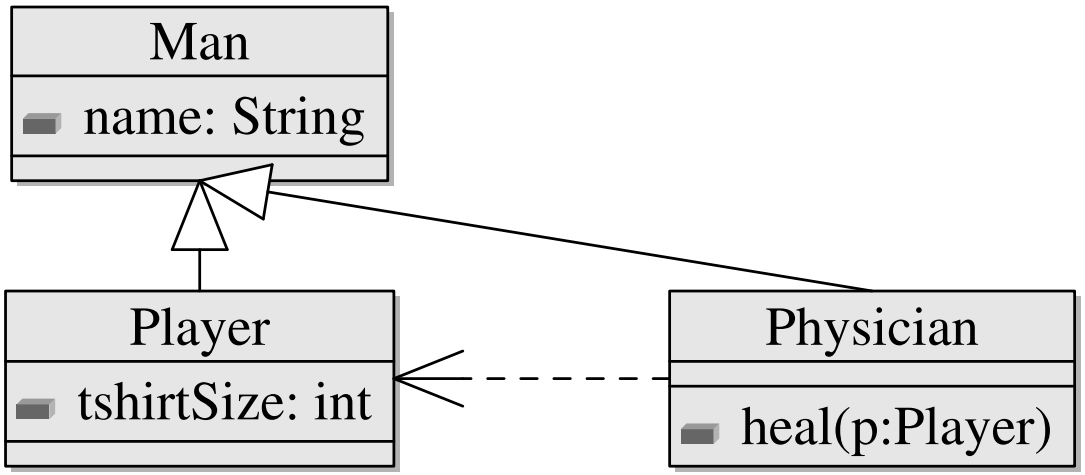


Universitatea
Politehnica
Bucureşti

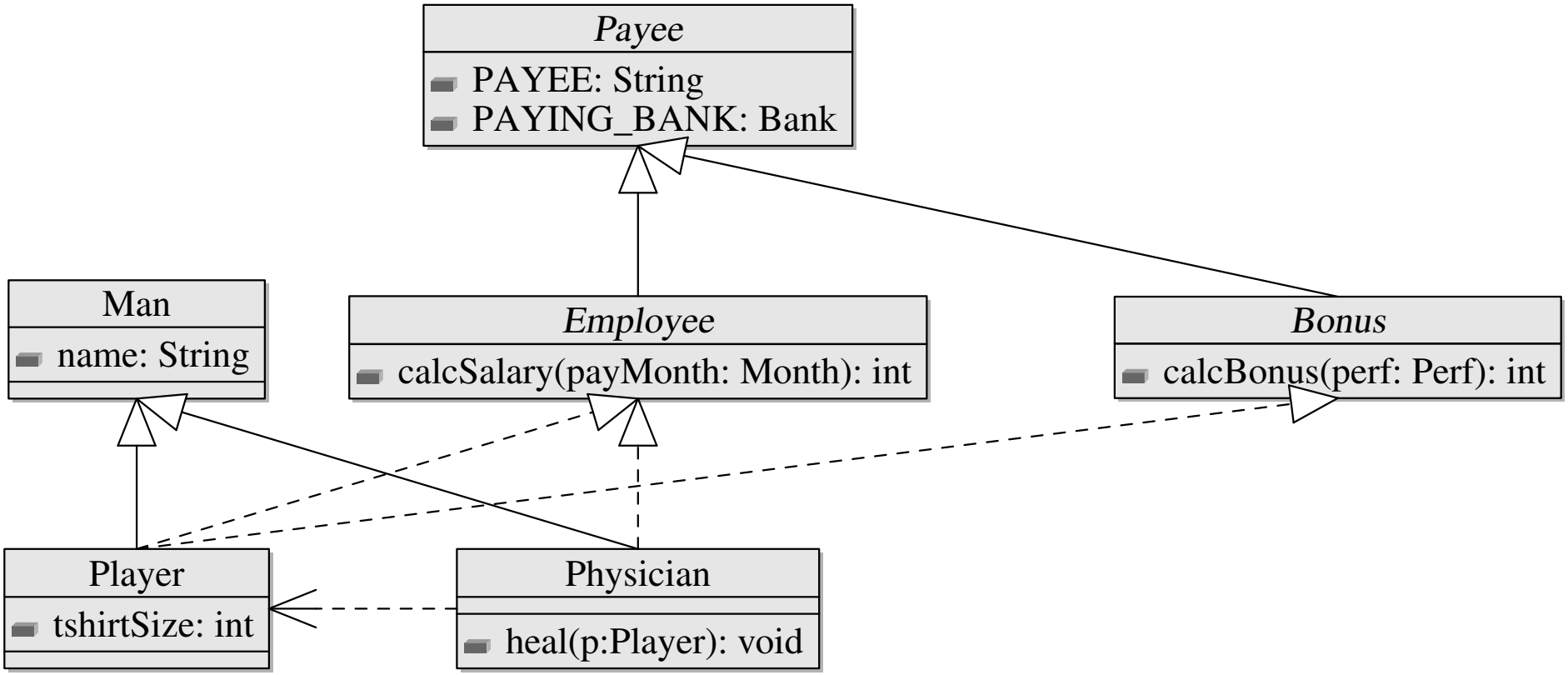
Curs 2: ce contine o clasa



Curs 3: relatia dintre clase



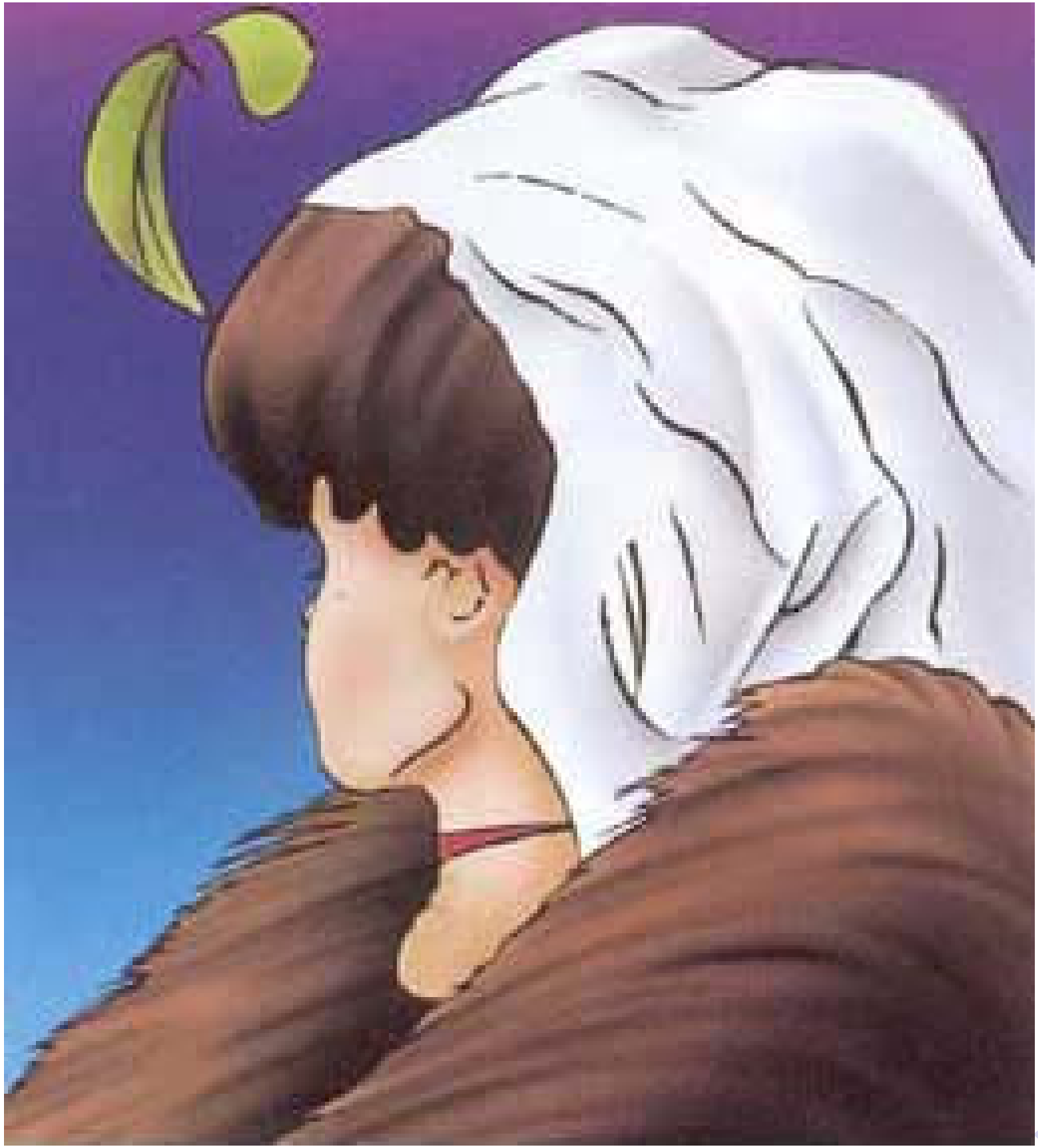
Curs 4: clase abstracte si interfete



Exercitii retrospective

- ▸ Quizes

Polimorfism



Definitia polimorfismului

Definition

Polimorfismul este caracteristica unei entitati de a avea forme diferite (structural, comportamental etc.) in contexte diferite.

Spre exemplu, in OOP, obiectele au aceasta caracteristica

Polimorfism in OOP

```
class Vehicle
interface Diesel
interface Electric
interface PublicTransport
class Hybrid extends Vehicle implements Electric, Diesel
class Bus extends Vehicle implements Diesel, PublicTransport
```

- A Hybrid IS-A Vehicle
- A Hybrid IS-A Electric
- A Hybrid IS-A Diesel
- A Hybrid IS-A Hybrid
- A Hybrid IS-A Object

Polimorfism in OOP

```
class Vehicle
interface Diesel
interface Electric
interface PublicTransport
class Hybrid extends Vehicle implements Electric, Diesel
class Bus extends Vehicle implements Diesel, PublicTransport
```

- A Bus IS-A Vehicle
- A Bus IS-A Diesel
- A Bus IS-A PublicTransport
- A Bus IS-A Bus
- A Bus IS-A Object

Polimorfism in OOP

```
class Vehicle
interface Diesel
interface Electric
interface PublicTransport
class Hybrid extends Vehicle implements Electric, Diesel
class Bus extends Vehicle implements Diesel, PublicTransport
```

Daca avem acelasi membru / metoda in mai multe "fatete" ale unui obiect, care se foloseste?

Dynamic polymorphism (overriding)

Late binding, or dynamic binding is a computer programming mechanism in which the method being called upon an object or the function being called with arguments is looked up by name at runtime

```
public class Bike {
    public void sound() { System.out.println("Swoosh"); }
}
public class MotorBike extends Bike {
    public void sound() { System.out.println("Grrrr"); }
}
...
    public static void main(String args[]){
        Bike obj=new MotorBike();
        obj.sound();
    }
```

private, final and static methods and variables uses static binding

Functii virtuale

In C++, se va face dynamic binding (la run-time) doar pentru functiile marcate ca fiind virtuale

Daca o functie este supraincarcata fara a fi marcata drept virtuala, se va face static binding (la compilare)

▸ Despre functii virtuale in C++

Dynamic polymorphism (overriding)

Atentie! dynamic binding se refera la metode, nu la membrii / campuri

```
public class Bike {
    public int topSpeed = 30;
}
public class MotorBike extends Bike {
    public int topSpeed = 150;
}
...
    public static void main(String args[]){
        Bike obj=new MotorBike();
        System.out.println(obj.topSpeed);
    }
```

La afisare va fi folosit membrul din referinta (in acest caz superclasa).

Dynamic polymorphism (overriding)

Atentie! dynamic binding se refera la metode, nu la membrii / campuri

```
public class Bike {
    public int topSpeed = 30;
}
public class MotorBike extends Bike {
    // public int topSpeed = 150;
    public MotorBike() {
      topSpeed = 150;
    }
}
...
    public static void main(String args[]){
        Bike obj=new MotorBike();
        System.out.println(obj.topSpeed);
    }
```

Daca subclasa nu defineste membrul cu acelasi nume ca in superclasa, in constructor se va folosi membrul din superclasa (mai putin daca acesta este private)

Dynamic polymorphism (overriding)

Atentie! Overloading se rezolva la compile-time (fiecare metoda capata un nume unic) - vezi Static Polymorphism

```
public class Road extends Path {}
public class MotorBike{
    public void ride(Road r) {
        System.out.println("Grrr on road");
    }
    public void ride(Path p) {
        System.out.println("Grrr on path");
    }
}
...
 public static void main(String args[]){
    MotorBike m = new MotorBike();
    Path r = new Road();
    m.ride(r);
 }
```

Double dispatch

The problem with Single Dispatch

```
public class Path {}
public class Road extends Path {}
public class Bike {
    public void ride(Road r) { sout("Swoosh on road"); }
    public void ride(Path p) { sout("Swoosh on path"); }
}
public class MotorBike extends Bike{
    public void ride(Road r) { sout("Grrr on road"); }
    public void ride(Path p) { sout("Grrr on path"); }
}
...
 public static void main(String args[]){
    Bike b = new Bike(); Bike m = new MotorBike();
    Path p = new Path(); Path r = new Road();
    b.ride(p); b.ride(r); m.ride(p); m.ride(r);
 }
```

Double dispatch

Mixing the second Dispatch on top of Single Dispatch

```
public class Path {
    public void echo(Bike b){ b.ride(this); }
}
public class Road extends Path {
    public void echo(Bike b){ b.ride(this); }
}
public class Bike {
    public void ride(Road r) { sout("Swoosh on road"); }
    public void ride(Path p) { sout("Swoosh on path"); }
}
public class MotorBike extends Bike{
    public void ride(Road r) { sout("Grrr on road"); }
    public void ride(Path p) { sout("Grrr on path"); }
}
 public static void main(String args[]){
    Bike b = new Bike(); Bike m = new MotorBike();
    Path p = new Path(); Path r = new Road();
    // b.ride(p); b.ride(r); m.ride(p); m.ride(r);
    p.echo(b); r.echo(b); p.echo(m); r.echo(m);
```

Static / Ad-hoc polymorphism (overloading)

Ad-hoc polymorphism (...) allows a polymorphic value to exhibit different behaviors when “viewed” at different types. The most common example of ad-hoc polymorphism is overloading, which associates a single function symbol with many implementations; the compiler (or the runtime system, depending on whether overloading resolution is static or dynamic) chooses an appropriate implementation for each application of the function, based on the types of the arguments.
B. Pierce, "Types and Programming Languages", MIT Press

Static / Ad-hoc polymorphism (overloading)

Static = compile time

```
1 + 2 = 3                          // integer addition
3 + 0.1415 = 3.1415                // float addition
[1,2,3] + [4,5] = [1,2,3,4,5]      // list concatenation
"bab" + "oon" = "baboon"           // string concatenation
red + yellow = orange              // custom class operator+
```

Parametric polymorphism / Generics

Parametric polymorphism (...), allows a single piece of code to be typed "generically", using variables in place of actual types, and then instantiated with particular types as needed. Parametric definitions are uniform: all of their instances behave the same. (...)
B. Pierce, "Types and Programming Languages", MIT Press

Parametric polymorphism / Generics

o singura implementare (structura de date, algoritm) pentru mai multe tipuri de date

```
List<Player> team = new ArrayList<Player>();
List<Integer> grades = new ArrayList<Integer>();
```

Parametric polymorphism / Generics

o singura implementare (structura de date, algoritm) pentru mai multe tipuri de date

```
List<Player> team = new ArrayList<Player>();
List<Integer> grades = new ArrayList<Integer>();

Collections.sort(team);
Collections.sort(grades);
```

Clase interne (Nested Classes)

Nevoia de clase interne

Apar situatii in care:

- o clasa este folosita intr-un singur loc in program (in interiorul altei clase)
- o parte a continutului unei clase ar fi mai clar definita drept o alta clasa (dar are inca nevoie de acces la clasa parinte)

Ce este o clasa interna?

O clasa interna (Nested Class) se defineste in interiorul unei alte clase, denumit clasa externa (Outer Class),

Ce este o clasa interna?

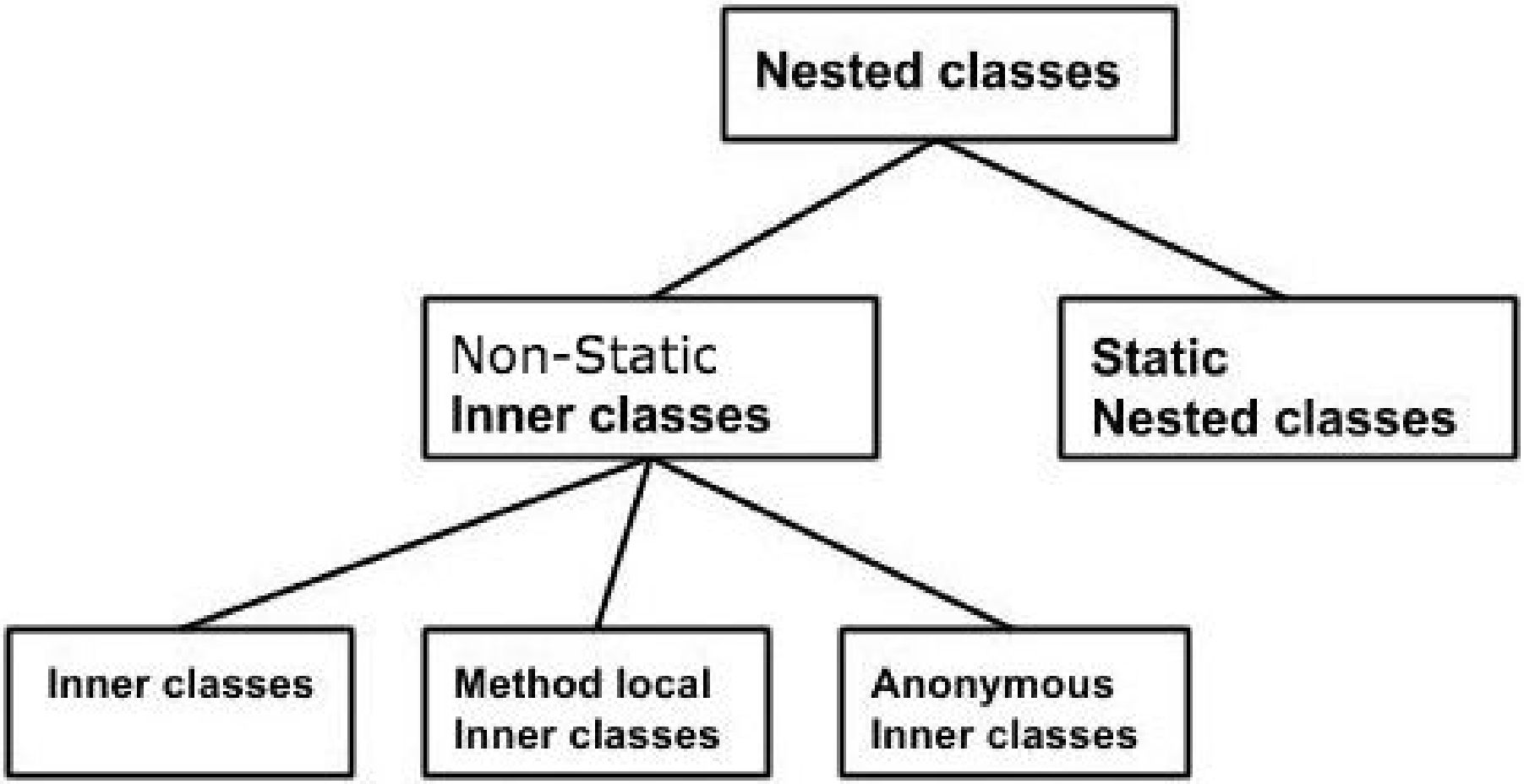
O clasa interna (Nested Class) se defineste in interiorul unei alte clase, denumit clasa externa (Outer Class), in diferite forme (ca membru, in interiorul unei metode, etc.).

Ce este o clasa interna?

O clasa interna (Nested Class) se defineste in interiorul unei alte clase, denumit clasa externa (Outer Class), in diferite forme (ca membru, in interiorul unei metode, etc.).

O clasa care nu este definita in interiorul unei alte clase se numeste Top-Level Class.

Tipuri de clase interne



▸ Inner Classes Tutorial

Inner Classes (clase interne normale)

Poate fi vizibila in exterior, caz in care poate fi accesata prin intermediul unei instante a clasei externe (ori ca rezultat al unei metode ori prin instantiere)

```
public class Person {
    public class Account {
        public String bank;
        public String iban;
    }
    public Account getAccount() {
        Account c = new Account("SwissBank", "CH23...");
        return c;
    }
}
public class Main {
    public static void main(String[] args) {
        Person pl = new Person();
        Person.Account c1 = pl.getAccount();
        Person.Account c2 = pl.new Account("USBank", "US23..
            ");
    }
```

Inner Classes (clase interne normale)

Putem sa accesam instanta clasei externe care a creat-o

```
public class Person {
    private name;
    public class Account {
        public String bank;
        public String iban;
        public String toString {
            return Person.this.name+"'s account: "+iban;
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Person pl = new Person("Mike");
        Person.Account c = pl.new Account("USBank", "US23.."
            );
        System.out.println(c); // ce se afiseaza?
    }
}
```

Inner Classes (clase interne normale)

Poate fi 'private', caz in care implementarea este ascunsa, dar interfata poate fi publica

```
interface BankAccount {
}
public class Person {
    private class Account implements BankAccount {
        public String bank;
        public String iban;
    }
    public BankAccount getAccount() {
        Account c = new Account("SwissBank", "CH23...");
        return c;
    }
}
public class Main {
    public static void main(String[] args) {
        Person pl = new Person();
        BankAccount c3 = pl.getAccount(); // ok
    }
}
```

Method Local Inner Classes (clase interne in metode si blocuri)

Vizibilitatea clasei se limiteaza la metoda, respectiv bloc (desi codul binar se creeaza oricum). Pentru a putea folosi variabilele si parametrii metodei, clasa trebuie sa fie final.

```
public class TestMethodLocalInnerClass{
    public static void main(String[] args){
        class Greeter implements Runnable{
            private final String _greeted;
            public Greeter(String greeted){
                super();
                _greeted = greeted;
            }
            public void run(){
                System.out.printf("Hello %s!\n", _greeted);
            }
        }

        new Greeter("world").run();
        new Greeter("dog").run();
```

Anonymous Inner Classes (clase anonime)

Daca o clasa este instantiata intr-un singur loc si apoi folosita (prin upcasting) doar prin intermediul clasei de baza sau al interfetei, numele sau nu mai este important.

```
new Thread( new Runnable() {
    Overridepublic void run() // do magic on separate thread).start();
```

▸ More about Runnable

Anonymous Inner Classes (clase anonime)

Daca o clasa este instantiata intr-un singur loc si apoi folosita (prin upcasting) doar prin intermediul clasei de baza sau al interfetei, numele sau nu mai este important.

```
public class MyClass extends Applet {
    ...
        someObject.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                ...//Event listener implementation goes here
                    ...
            }
        });
    ...
    }
}
```

▸ More about Event Listeners

Anonymous Inner Classes (clase anonime)

Limitari:

- O clasa anonima poate sa extinda o singura clasa sau sa implementeze o singura interfata
- O clasa anonima nu are constructori (nu are nume, nu am sti cum se cheama constructorul); in mod implicit se invoca constructorul din clasa de baza care se potriveste cel mai bine cu parametrii folositi la initializare (sau constructorul default al clasei de baza daca nu sunt folositi parametrii)

Static Nested Classes (clase interne statice)

O clasa interna poate fi un membru static al clasei externe:

- nu va avea acces la instantele clasei externe (Outer.this), ci functioneaza ca o alta clasa top-level
- are sens sa fie interna pentru ca este folosita doar prin clasa externa (packaging convenience)
- nu avem nevoie de o instanta a clasei externe pentru a creea o instanta a clasei interne

```
LinkedList.Entry, Map.Entry
```

▸ How to use it

De citit

Este util sa cititi textul laboratorului inainte de laborator:

- Lab 5: Clase Interne

- Tutorial: Double Dispatch